

Task 04 — Inter-Service Communication

Step 1

REST Communication

REST is how one service communicates with another using HTTP.

Example:

Order Service → GET → User Service

HTTP

You will use HTTP methods such as:

GET

POST

PUT

DELETE

The main communication is:

GET /api/users/{id}

JSON

The services exchange data in JSON format.

For example, User Service can return:

```
{  
  "id": 1,  
  "name": "Ravi",  
  "email": "ravi@gmail.com"  
}
```

Order Service receives this JSON response.

Synchronous Communication

This is the main communication you will implement.

Example:

Order Service

|

| Request

↓

User Service

|

| Response

↓

Order Service

Order Service **waits for User Service's response.**

Asynchronous Communication

Here the service doesn't wait for an immediate response.

For example:

Order Service

|

| Event

↓

Message Broker

|

↓

User/Payment/Notification Service

Examples of technologies are Kafka or RabbitMQ.

Mainly need to implement synchronous REST communication and document asynchronous communication.

Step 2: Create

Create these two projects:

```
microservices
|
├── user-service
└── order-service
```

User Service

Run it on:

localhost:8081

Project structure:

```
user-service
└── src/main/java/com/example/userservice
    ├── UserServiceApplication.java
    ├── controller
    |   └── UserController.java
    ├── service
    |   └── UserService.java
    └── model
        └── User.java
```

Step 3 — Create User API

GET /api/users/{id}

User.java

```
package com.example.userservice.model;
```

```
public record User (Long id, String name, String email) {
}
```

UserService.java

```
package com.example.userservice.service;

import com.example.userservice.model.User;
import org.springframework.stereotype.Service;
import java.util.Map;

public class UserService {

    private final Map<Long, User> users = Map.of(
        1L, new User(1L, "John", "john@gmail.com"),
        2L, new User(2L, "Alice", "alice@gmail.com"));

    public User getUserById(Long id) {
        return users.get(id);
    }
}
```

UserController.java

```
package com.example.userservice.controller;

import com.example.userservice.model.User;
import com.example.userservice.service.UserService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

public class UserController {

    private final UserService userService;

    public UserController(UserService userService) {
        this.userService = userService;
    }

    public ResponseEntity<User> getUser(@PathVariable Long id) {
        User user = userService.getUserById(id);
    }
}
```

```

if (user == null) {
    return ResponseEntity.notFound().build();
}
return ResponseEntity.ok(user);
}
}

```

application.properties

```

spring.application.name=user-service
server.port=8081

```

Step 4 — Create Order API

Create the second Spring Boot project:

order-service

Structure:

order-service

└─ src/main/java/com/example/orderservice

├─ OrderServiceApplication.java

├─ controller

└─ OrderController.java

├─ service

└─ OrderService.java

├─ client

└─ UserClient.java

└─ model

├─ Order.java

└─ UserResponse.java

GET /api/orders/{id}

Order.java

```
package com.example.orderservice.model;  
  
public record Order( Long id, Long userId, String status, double amount) {  
  
}
```

UserResponse.java

```
package com.example.orderservice.model;  
  
public record UserResponse(Long id, String name, String email) {  
  
}
```

Step 5 — Make Order Service call User Service

The flow should be:

GET /api/orders/101



Order Service

| GET /api/users/1



User Service



User JSON



Order Service



Client

Create UserClient.java.

```
package com.example.orderservice.client;

import com.example.orderservice.model.UserResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

public class UserClient {

    private final RestClient restClient;

    public UserClient(RestClient.Builder builder) {

        this.restClient = builder

            .baseUrl("http://localhost:8081")

            .build();

    }

    public UserResponse getUser(Long userId) {

        return restClient.get()

            .uri("/api/users/{id}", userId)

            .retrieve()

            .body(UserResponse.class);

    }

}
```

Now Order Service can communicate with User Service.

Step 6 — Implement Request, Response, Timeout and Error

Request

When somebody calls:

```
GET http://localhost:8082/api/orders/101
```

Order Service finds the userId.

Example:

Order 101

User ID = 1

Then it calls:

GET <http://localhost:8081/api/users/1>

Response

User Service returns:

```
{  
  "id": 1,  
  "name": "John",  
  "email": "john@gmail.com"  
}
```

Create:

UserResponse.java

```
package com.example.orderservice.model;  
  
public record UserResponse(Long id,String name,String email) {  
  
}
```

Order Service

Create:

Order.java

```
package com.example.orderservice.model;  
  
public record Order(Long id,Long userId, String status,double amount) {  
  
}
```

Then:

OrderService.java

```
package com.example.orderservice.service;

import com.example.orderservice.client.UserClient;
import com.example.orderservice.model.Order;
import com.example.orderservice.model.UserResponse;
import org.springframework.stereotype.Service;
import java.util.Map;

public class OrderService {

    private final UserClient userClient;

    private final Map<Long, Order> orders = Map.of(
        101L, new Order(101L, 1L, "CONFIRMED", 1500.00),
        102L, new Order(102L, 2L, "CREATED", 850.00)
    );

    public OrderService(UserClient userClient) {
        this.userClient = userClient;
    }

    public String getOrder(Long orderId) {
        Order order = orders.get(orderId);

        if (order == null) {
            return "Order not found";
        }

        UserResponse user = userClient.getUser(order.userId());
        return ""

            Order ID: %d

            User ID: %d

            User Name: %s
```

```
        User Email: %s
        Status: %s
        Amount: %.2f
        """".formatted(
            order.id(),
            order.userId(),
            user.name(),
            user.email(),
            order.status(),
            order.amount()
        );
    }
}
```

Order Controller

```
package com.example.orderservice.controller;

import com.example.orderservice.service.OrderService;
import org.springframework.web.bind.annotation.*;

public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }

    public String getOrder(@PathVariable Long id) {
        return orderService.getOrder(id);
    }
}
```

Now this API works:

GET <http://localhost:8082/api/orders/101>

Error Response

Create:

RemoteServiceException.java

```
package com.example.orderservice.exception;

public class RemoteServiceException extends RuntimeException {

    private final int status;

    public RemoteServiceException(int status, String message) {
        super(message);
        this.status = status;
    }

    public int getStatus() {
        return status;
    }

}
```

Update UserClient.java:

```
package com.example.orderservice.client;

import com.example.orderservice.exception.RemoteServiceException;
import com.example.orderservice.model.UserResponse;
import org.springframework.stereotype.Component;
import org.springframework.web.client.ResourceAccessException;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;

public class UserClient {
```

```

private final RestClient restClient;

public UserClient(RestClient.Builder builder) {

    this.restClient = builder

        .baseUrl("http://localhost:8081")

        .build();
}

public UserResponse getUser(Long userId) {
    try {
        return restClient.get()

            .uri("/api/users/{id}", userId)

            .retrieve()

            .onStatus(
                status -> status.value() == 404,
                (request, response) -> {
                    throw new RemoteServiceException(
                        404,
                        "User does not exist: " + userId );
                })
            .onStatus(
                status -> status.is5xxServerError(),
                (request, response) -> {
                    throw new RemoteServiceException(
                        503,
                        "User Service returned an error");
                })
    }
}

```

```

        .body(UserResponse.class);
    } catch (RemoteServiceException e) {
        throw e;
    } catch (ResourceAccessException e) {

        throw new RemoteServiceException(
            503,
            "User Service unavailable or request timed out");
    } catch (RestClientException e) {
        throw new RemoteServiceException(
            503,
            "Unable to communicate with User Service");
    }
}
}
}

```

Error in Order Controller

```

package com.example.orderservice.controller;

import com.example.orderservice.exception.RemoteServiceException;
import com.example.orderservice.service.OrderService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

public class OrderController {

    private final OrderService orderService;

    public OrderController(OrderService orderService) {
        this.orderService = orderService;
    }
}

```

```

public ResponseEntity<String> getOrder(@PathVariable Long id) {
    return ResponseEntity.ok(
        orderService.getOrder(id));
}

public ResponseEntity<ApiError> handleRemoteServiceError(
    RemoteServiceException exception) {
    return ResponseEntity
        .status(exception.getStatus())
        .body(new ApiError(exception.getStatus(),exception.getMessage()));
}

public record ApiError(int status,String message) {
}
}

```

Timeout

In application.properties:

```

spring.application.name=order-service
server.port=8082
user-service.base-url=http://localhost:8081
spring.http.client.connect-timeout=2s
spring.http.client.read-timeout=3s

```

The idea is:

Connect timeout = 2 seconds

Read timeout = 3 seconds

Step 7 — Test

User exists

Start both services.

User Service

8081

Order Service

8082

Postman:

GET <http://localhost:8082/api/orders/101>

Order 101 belongs to User 1.

Expected:

200 OK

Example:

Order ID: 101

User ID: 1

User Name: John

User Email: john@gmail.com

Status: CONFIRMED

Amount: 1500.00

This proves:

Order Service → User Service → Success

User does not exist

Add an order with a nonexistent user:

103L, new Order(103L, 999L, "CREATED", 450.00)

So: Order 103

└─ User ID = 999

Call:

GET http://localhost:8082/api/orders/103

Order Service calls:

GET http://localhost:8081/api/users/999

User Service responds:

404 Not Found

Order Service should return:

```
{  
  "status": 404,  
  "message": "User does not exist: 999"  
}
```

User Service unavailable

Now stop User Service.

Keep Order Service running.

User Service	STOPPED
Order Service	RUNNING

Call:

GET http://localhost:8082/api/orders/101

Order Service attempts:

Order Service



User Service

X

Unavailable

Expected:

503 Service Unavailable

Example:

```
{  
  "status": 503,  
  "message": "User Service unavailable or request timed out"  
}
```

Step 8 — Document synchronous vs asynchronous

Create a file:

communication.md

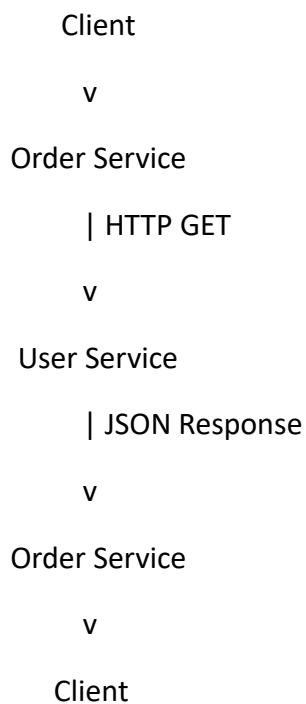
Put this in it:

Synchronous vs Asynchronous Communication

Synchronous Communication

In synchronous communication, the calling service waits for the response from the target service.

Example:



Advantages

- Simple to implement
- Immediate response
- Easy to understand
- Suitable when the caller needs the result immediately

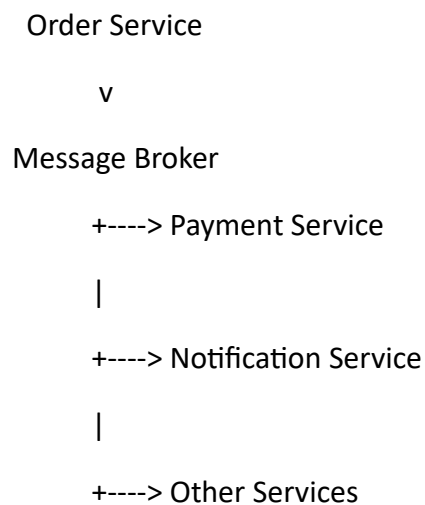
Disadvantages

- Caller depends on target service availability
- Network timeout can occur
- Failure of the target service can affect the request

Asynchronous Communication

In asynchronous communication, the calling service sends an event/message and does not wait for an immediate response.

Example:



Examples of message brokers:

- Apache Kafka
- RabbitMQ

Advantages

- Loose coupling
- Better resilience
- Services can process messages independently

Disadvantages

- More complex
- Requires message broker infrastructure
- Responses are not immediate
- Eventual consistency may occur

This task implements synchronous communication using REST/HTTP.

Asynchronous communication is documented for comparison.

Final deliverables

Two Spring Boot Services

user-service

└─ User API

GET /api/users/{id}

order-service

└─ Order API

GET /api/orders/{id}

Recommended ports:

User Service → 8081

Order Service → 8082

Inter-Service REST Call

Client

| GET /api/orders/101



Order Service

| GET /api/users/1



User Service

| JSON Response



Order Service



Client

This proves that **Order Service communicates with User Service using REST/HTTP.**

Error Handling

User exists

GET /api/orders/101



200 OK

User doesn't exist

GET /api/orders/103



404 Not Found

User Service unavailable / timeout

User Service stopped



GET /api/orders/101



503 Service Unavailable

The application should return a proper JSON error instead of crashing.

Postman Collection

Inter-Service Communication

├─ User Service

│ ├── Get Existing User

│ └─ Get Missing User

|

└─ Order Service

│ ├── Get Order - User Exists

│ ├── Get Order - User Doesn't Exist

│ └─ Get Order - User Service Unavailable